

Security review — 2026-09

Milestone 3 evidence: "Security review report (findings + applied fixes)", for the acceptance criterion "No open P1 security findings after remediation". It compiles findings already closed in code, each with its date, the file that fixes it and the test that proves the fix, plus a dependency scan and static analysis. Review run on 2026-09-07, updated through 2026-09-21.

Scope

- `apps/api` — the API: 87 mounted routes across 18 router files at review time.
- `packages/cardano` — the `simulated` and real adapters of the anchoring port (`AnchorPort`).
- `packages/shared` — the shared Zod contract.
- `contracts/` — the Aiken validators (Plutus V3).
- Dependencies of the whole workspace (`pnpm audit`).

Out of scope: infrastructure pentesting (Render, Turso, Blockfrost) beyond what the code controls; social engineering; the Cardano client (Lucid Evolution) as a library in itself.

Method

1. **Declared permission matrix** — `apps/api/test/route-guards.test.ts` rebuilds, by reading the routers already mounted in Express, the guard chain of every route and compares it against a literal. It was verified by breaking it (three deliberate mutations, all three red, reverted), not just by seeing it green.
2. **Manual audit of the third pattern** — routes that authorized inside the handler instead of in the route signature. Now declared in the signature (`authorize({ roles, acceso })`). Verified by neutralizing the guard: 7 of 13 rejection tests in `test/require-ownership.test.ts` turn red.
3. **Dependency scan** — `pnpm audit` run on 2026-09-07 against the 750 dependencies of the workspace, with every finding traced to its real chain (`paths`) to tell runtime dependencies apart from install-time or build/test ones. **Since 2026-09-11 it runs on every CI run** (`.github/workflows/ci.yml` → `job audit`; until 2026-10-07, inside `app`): one step that never fails prints the full report (the visibility of what is already triaged below, findings 9 and 10), and another one — the real gate — fails on **critical**, currently zero.
4. **Timing side channels** — measured, not estimated, with `test/auth-timing.test.ts` (9 interleaved runs, compared by order of magnitude).
5. **Static analysis (Semgrep)** — `p/security-audit` + `p/owasp-top-ten`, run locally on 2026-09-07 and wired into CI as its own job, in parallel with the others. Verified to actually catch issues, not just to pass: a synthetic reflected XSS (`res.send(`

...`${req.query.q}`

)) is detected by two rules and exits with code 1.

Findings

#	Finding	Severity	Status	Closed	Evidence
1	<code>JWT_SECRET</code> fell back to a public literal (<code>"dev-secret"</code>) if the environment variable was missing or empty — anyone could forge an admin token	P1	Closed	2026-08-20	<code>requireJwtSecret()</code> throws at import time if there is no secret; <code>test/jwt.test.ts</code>
2	<code>GET /evidence/:bundleId/proof/:fileHash</code> and <code>/:bundleId/files</code> lacked the second authorization layer — any authenticated user could read the Merkle root and file names of someone else's bundle by knowing its id	P1	Closed	2026-09-03	project-membership check via the bundle; <code>test/evidence-anchor.test.ts</code> → "a developer without membership ... gets 403"
3	<code>GET /developer/audit-log</code> returned the entire <code>AuditLog</code> , not scoped to the developer's projects, with <code>actorName / actorRole</code> of every user	P1	Closed	2026-09-04	<code>auditScope</code> , fail-closed, applied consistently with the rest of the permission matrix
4	Timing oracle on <code>/auth/login</code> : a non-existent email answered ~81 ms faster than a valid one (reveals whether an account exists)	P2	Closed	2026-08-20	always compares against <code>HASH_DUMMY</code> ; measured in <code>test/auth-timing.test.ts</code> (difference from ~81 ms down to ~0.7 ms)
5	9 investor routes authorized ownership (<code>unitId / investorId</code>) with an <code>if</code> copied inside the handler, with no shape in the signature and no compiler protection	P2	Closed	2026-09-04	ownership rule declared in the route signature; <code>test/require-ownership.test.ts</code>
6	<code>POST/PATCH /users</code> used a local <code>z.enum</code> without <code>notary</code> , breaking the shared-schema rule — an admin could not create or promote a notary through the API	P3	Closed	2026-09-03	<code>userRoleSchema</code> from <code>@plataforma/shared</code> ; <code>test/users-roles.test.ts</code>
7	<code>signToken / verifyToken</code> did not pin <code>algorithm / algorithms</code> explicitly — they relied on the <code>jsonwebtoken</code> default to restrict to HS*. No real hole (the default already did it with a string secret), but an implicit dependency	P3	Closed	2026-09-04	explicit HS256 on both sides; <code>test/jwt.test.ts</code> → "the signing algorithm is pinned"
8	<code>qs@6.15.3</code> (a real dependency of <code>express / body-parser</code> , on every request's path) vulnerable to DoS via array-limit bypass and <code>isBuffer</code>	P3 (moderate, CVSS 5.3)	Closed	2026-09-07	override to <code>qs@6.16.0</code> in <code>package.json</code> ; <code>pnpm audit</code> went from 10 to 8 moderate
9	<code>tar@6.2.1</code> with 1 critical + 7 high — the chain was <code>bcrypt</code> → <code>@mapbox/node-pre-gyp</code> → <code>tar</code>	Critical/High, not exploitable at runtime	Closed	2026-09-10	<code>bcrypt@6.0.0</code> replaced <code>@mapbox/node-pre-gyp</code> with <code>node-gyp-build / node-addon-api</code> and took the whole <code>tar</code> chain with it. <code>pnpm audit</code> no longer reports it

#	Finding	Severity	Status	Closed	Evidence
10	<code>browserslist</code> , <code>nanoid</code> , <code>undici</code> , <code>vitest/@vitest/mocker</code> , <code>postcss</code> , <code>baseline-browser-mapping</code> — currently 4 high + 8 moderate, all in the <code>vite/vitest/@babel/@tanstack/router-plugin</code> chain (build and test)	High/Moderate, not exploitable at runtime	Open, no immediate plan	—	none of these packages is served in the production bundle or runs in the deployed API process; they resolve with the next bump of the build tools and do not warrant a manual override. It is exactly the report the non-blocking <code>pnpm audit</code> step prints on every CI run
11	No revocation of an individual JWT (valid 7 days; can only be cut by deactivating the whole account)	— (product decision, not a finding)	Open on purpose	—	infrastructure for a problem that does not hurt yet; mitigated because <code>authenticate()</code> re-checks <code>isActive</code> on every request
12	9 GitHub Actions references by mutable tag (<code>@v7</code> , <code>@v6</code> , <code>@v1</code>) in <code>ci.yml</code> — a re-pointed tag in the Action's repo is a supply-chain risk, not a bug in this code	Low (hardening, not an active vulnerability)	Closed	2026-09-07	pinned to their commit SHA, with the tag as a comment for readability
13	Semgrep flags 3 pnpm settings (<code>blockExoticSubdeps</code> , <code>minimumReleaseAge</code> , <code>trustPolicy</code>) and 1 npm setting (<code>min-release-age</code>) as missing	Low (hardening)	Open on purpose	—	the three pnpm settings need pnpm 10.16+/10.21+/10.26+ and the repo is pinned to <code>pnpm@9.15.0</code> — adding them today would be a setting the real pnpm silently ignores. The npm one does not apply: the repo does not install with npm. Explicitly excluded in the CI job (<code>--exclude-rule</code>), not ignored by default
14	Semgrep (<code>gha-curl-pipe-shell</code>) flagged the login in <code>.github/workflows/reconcile.yml: curl ... python3 -c</code> . The pipe carried data (JSON) into a literal script, not code downloaded from the network, but it has the shape of the <code>curl sh</code> pattern	Low (false positive by shape)	Closed	2026-09-21	the response is stored in a variable and parsed separately; a <code>nosemgrep</code> marker did not work because the finding is reported on the whole <code>run: block</code> . Verified with Semgrep 1.176.1 and the CI rulesets: 1 finding before, 0 after

Conclusion

Zero open P1 findings. The three P1s found (`JWT_SECRET` , the second authorization layer on evidence, the unscoped audit log) are closed in code, each with a test that reproduces the original bug and proves the fix. `pnpm audit` runs on every CI run since 2026-09-11: the real gate is **zero critical**, currently green — the only critical there ever was (`tar` , via `bcrypt`) was closed by the bump to `bcrypt@6.0.0` . The remaining high/moderate dependency findings are confined to install and build paths that a client of the deployed API cannot reach — they stay documented and open as hygiene debt, not as production risk, and the non-blocking CI step reports them on every run. The only dependency finding with real runtime exposure (`qs` , moderate) was closed the same day. Static analysis (Semgrep) runs in CI and found no vulnerable code: the 16 initial findings were 2 false positives (suppressed with `nosemgrep` , with the reason documented in the code) and 14 supply-chain hardening items, of which the GitHub Actions pinning was closed and the `pnpm/npm` settings are deferred for version incompatibility (finding 13).

Reproducing this review

```
pnpm --filter @plataforma/api test route-guards.test.ts require-ownership.test.ts \
  auth-timing.test.ts jwt.test.ts users-roles.test.ts evidence-anchor.test.ts
pnpm audit
pip install semgrep && semgrep scan --config=p/security-audit --config=p/owasp-top-ten \
  --exclude-rule package_managers.pnpm.pnpm-block-exotic-sub-dependencies.pnpm-block-exotic-
  sub-dependencies \
  --exclude-rule package_managers.pnpm.pnpm-missing-minimum-release-age.pnpm-minimum-release-
  age \
  --exclude-rule package_managers.pnpm.pnpm-trust-policy.pnpm-trust-policy \
  --exclude-rule package_managers.npm.npm-missing-minimum-release-age.npm-missing-minimum-
  release-age \
  --exclude contracts/build --exclude specs/evidencia-m3/2-api/postman \
  --exclude apps/api/api-docs/propnexus.postman_collection.json --error
```